



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Практическая работа № 5/4 ч.

Разработка мобильных приложений

	<i>(наименование дисциплины (модуля) в соответствии с учебным планом)</i>
Уровень	бакалавриат
	<i>(бакалавриат, магистратура, специалитет)</i>
Форма обучения	очная
	<i>(очная, очно-заочная, заочная)</i>
Направление(-я) подготовки	09.03.02 «Информационные системы и технологии»
	<i>(код(-ы) и наименование(-я))</i>
Институт	кибербезопасности и цифровых технологий
	<i>(полное и краткое наименование)</i>
Кафедра	КБ-14 «Цифровые технологии обработки данных»
	<i>(полное и краткое наименование кафедры, реализующей дисциплину (модуль))</i>

Используются в данной редакции с учебного года

2024/25

(учебный год цифрами)

Проверено и согласовано « ____ » ____ 20 ____ г.

*(подпись директора Института/Филиала
с расшифровкой)*

Москва 2024 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	3
1 RETROFIT	5
2 PICASSO	13
3 КОНТРОЛЬНОЕ ЗАДАНИЕ.....	15

ВВЕДЕНИЕ

Сетевое взаимодействие является ключевой частью работы большинства современных Android-приложений, им необходим доступ к данным из удаленных источников, например из веб-сервисов или облачных хранилищ. Сетевое взаимодействие, как правило, осуществляется посредством использования протокола HTTP, который используется для передачи данных между клиентом (приложением) и сервером. Приложение отправляет запросы и получает ответы от сервера, которые могут содержать данные в различных форматах, чаще всего используется JSON (JavaScript Object Notation) - текстовый формат обмена данными.

Для организации взаимодействия между клиентом и сервером часто используется REST API (Representational State Transfer APIs также известный как RESTful API), это архитектурный стиль, который определяет набор правил для создания веб-сервисов, он основывается на принципах клиент-серверной архитектуры и использования стандартных методов HTTP.

Основные характеристики:

1. **Клиент-сервер:** Клиент и сервер функционируют независимо друг от друга, что позволяет их развивать и масштабировать отдельно.

2. **Статусный интерфейс:** Каждое взаимодействие между клиентом и сервером выполняется с помощью stateless запросов, что означает, что сервер не сохраняет состояние между запросами.

3. **Кэширование:** Ответы от сервера могут быть закэшированы, чтобы уменьшить количество запросов и повысить производительность.

4. **Единообразие интерфейса:** для упрощения архитектуры все взаимодействия между клиентом и сервером осуществляются через стандартные методы HTTP. Основные HTTP-методы, используемые для работы с REST API, включают GET (получение данных с сервера), POST (отправка данных на сервер), PUT (обновление существующих данных) и DELETE (удаление данных). Например, GET-запрос может использоваться для получения списка пользователей, а POST - для добавления нового пользователя.

5. **Многоуровневая система:** Архитектура может быть многоуровневой, что позволяет добавлять промежуточные слои (например, прокси-серверы и шлюзы) для управления запросами.

К основным недостаткам

1. Избыточная и недостаточная выборка: REST API часто возвращают либо слишком много, либо слишком мало данных. Клиенты обычно получают все доступные поля ресурса, даже если им требуется только часть данных.

Такая избыточная выборка может привести к увеличению времени отклика и пропускной способности канала. И наоборот, недостаточная выборка возникает, когда клиенту приходится делать несколько запросов к различным endpoints, чтобы получить необходимые данные.

2. Меньшая гибкость: REST API обеспечивают ограниченную гибкость при запросах и манипуляциях с данными, поскольку используют структурированный, основанный на ресурсах подход с заранее определенными endpoints. Хотя такой подход более прост для понимания и реализации, он лишен гибкости, когда клиентам требуются более тонкие или детальные запросы.

3. Версионность: По мере роста и развития приложения управление изменениями в REST API может стать сложной задачей. Практика версионирования API различна; некоторые подходы могут привести к дублированию кода и головной боли при сопровождении.

4. Менее эффективен для сложных проектов: REST API может оказаться не лучшим выбором для приложений со сложными требованиями к данным и более сложными отношениями между ресурсами. При увеличении количества ресурсов и связей управление несколькими endpoints и вложенными данными может стать неуправляемым.

Обработка сетевых запросов в приложении является непростой задачей, так как для этого требуется работать с различными форматами данных, обрабатывать ошибки (например, отсутствие подключения к интернету), работать с потоками (запрос нельзя выполнять в основном потоке, так как это блокирует пользовательский интерфейс). Ручная реализация сетевых запросов через стандартные библиотеки Java, такие как `URLConnection`, требует написания большого объема кода, это усложняет код приложения и делает его менее читаемым. Для упрощения работы с сетевым взаимодействием и решения вышеперечисленных проблем возможно использовать специальные библиотеки, которые берут на себя функциональность по созданию, отправке запросов и обработке ответов. Среди популярных можно выделить Volley от компании Google, OkHttp и Retrofit от Square Open Source.

1 RETROFIT

Задание. Создать новый модуль. В меню «File | New | New Module | Phone & Tablet Module | Empty Views Activity». Название модуля «RetrofitApp». Необходимо реализовать приложение, позволяющее просматривать список дел и статус их выполнения.

Retrofit является популярной библиотекой для приложений под ОС Android, разработанной Square Open Source, которая упрощает работу с сетевым взаимодействием. Библиотека использует *OkHttp* для выполнения HTTP-запросов и поддерживает все основные методы протокола HTTP. Основные преимущества использования библиотеки заключаются в:

Возможность использования аннотаций для создания HTTP-запросов. Т.е. возможно описать API с помощью интерфейсов, где запросы определяются аннотациями, например `@GET("users")` описывает запрос для получения списка пользователей. При работе с API необходимо указать базовый URL – адрес сервера, с которым будет взаимодействовать приложение. Например, если базовый URL «`https://api.example.com/`», аннотация `@GET("users")` добавит к нему путь, создавая полный URL «`https://api.example.com/users`». Методы указывают тип HTTP-запроса и возвращают объект *Call*, представляющий запрос. Список основных аннотаций приведен в таблице ниже.

Аннотация	Описание
<code>@GET("users")</code>	GET-запрос для базового адреса. Также можно указать параметры в скобках
<code>@POST("users")</code>	POST-запрос для базового адреса. Также можно указать параметры в скобках
<code>@PUT("users/{id}")</code>	PUT-запрос по адресу users, включая значение id
<code>@PATCH("users/{id}")</code>	PATCH-запрос по адресу users, включая значение id
<code>@DELETE("users/{id}")</code>	DELETE-запрос по адресу users, включая значение id
<code>@Path("id")</code>	Переменная для замещения конечной точки, например, username подставится в {username} в адресе конечной точки
<code>@Query("page")</code>	Задаёт имя ключа запроса со значением параметра
<code>@Body</code>	Используется в POST-вызовах (из Java-объекта в JSON-строку)
<code>@Header</code>	Задаёт заголовок со значением параметра
<code>@Headers</code>	Задаёт все заголовки вместе
<code>@Multipart</code>	Используется при загрузке файлов или изображений
<code>@FormUrlEncoded</code>	Используется при использовании пары "имя/значение" в POST-запросах
<code>@FieldMap</code>	Используется при использовании пары "имя/значение" в POST-запросах
<code>@Url</code>	Для поддержки динамических адресов

Поддержка различных форматов данных. По умолчанию *Retrofit* поддерживает работу с текстовыми данными, но для более сложных форматов, таких как JSON или XML. Автоматическое преобразование данных между форматами, используемыми в HTTP-запросах и ответах, обеспечивают конвертеры. Они включают в себя процесс сериализации, который преобразует Java-объекты в формат, подходящий для отправки на сервер, и десериализации, который преобразует ответы сервера в объекты *Java*. Наиболее популярным является *Gson*, который используется для работы с JSON, он автоматически преобразует JSON-ответы сервера в объекты *Java* и обратно. Например, если сервер возвращает список пользователей в формате JSON, конвертер *Gson* преобразует этот JSON в список объектов *User*. При настройке *Retrofit* возможно указать, какой конвертер использовать. Поддерживаемый список конвертеров представлен ниже:

- Gson: `com.squareup.retrofit2:converter-gson`;
- Jackson: `com.squareup.retrofit2:converter-jackson`;
- Moshi: `com.squareup.retrofit2:converter-moshi`;
- Protobuf: `com.squareup.retrofit2:converter-protobuf`;
- Wire: `com.squareup.retrofit2:converter-wire`;
- Simple XML: `com.squareup.retrofit2:converter-simplexml`;
- JAXB: `com.squareup.retrofit2:converter-jaxb`;
- Scalars (primitives, boxed, and String):
`com.squareup.retrofit2:converter-scalars`.

Асинхронная работа. Запрос к серверу может выполняться как синхронно с помощью метода *execute()*, так и асинхронно с помощью метода *enqueue()*. Последний отправляет запрос в фоновом потоке, после завершения запроса передает результат обратно в основной поток, далее результат обрабатывается через метод *onResponse()*, который вызывается при успешном завершении запроса.

Обработка ошибок. Во время работы с сетью могут возникать различные проблемы, такие как отсутствие соединения или некорректные ответы от сервера. Существует несколько основных типов ошибок: ошибки соединения (приложение не может установить связь с сервером), ошибки протокола HTTP (ошибки клиента и ошибки сервера) и ошибки сериализации данных (данные от сервера не могут быть корректно преобразованы в объекты *Java*). Для обработки таких ошибок, которые возникают во время выполнения запроса, используется метод *onFailure()*. Если запрос выполнен успешно, но сервер вернул ошибку HTTP, вызывается метод *onResponse()*, где возможно проверить статус ответа с помощью *response.isSuccessful()*. Если этот метод возвращает *false*, можно обработать код ошибки и предоставить пользователю соответствующее сообщение.

В первую очередь необходимо добавить зависимости в *build.gradle*.

```
implementation("com.squareup.retrofit2:retrofit:2.11.0")  
implementation("com.squareup.retrofit2:converter-gson:2.11.0")
```

Для возможности обеспечения работы с сетевой частью в манифесте необходимо указать разрешение *android.permission.INTERNET*.

Для работы с Retrofit понадобятся три класса.

- *POJO (Plain Old Java Object)* или *Model Class* - json-ответ от сервера необходимо реализовать в виде модели данных;
- *Retrofit* - класс для обработки результатов. Необходимо указать базовый адрес в методе `baseUrl()`;
- *interface* - интерфейс для управления адресом сервера, используя команды GET, POST и т.д.

Реализация задачи сетевого взаимодействия с помощью Retrofit возможно декомпозировать на отдельные задачи.

1. Изучить структуру ответа сайта в виде JSON (или других форматов) и создать на его основе Java-класс в виде POJO.

В классе часто используются аннотации. Иногда они необходимы, иногда их возможно пропустить. В некоторых случаях аннотации помогают избежать ошибок.

В качестве API-сервера будет использоваться онлайн-сервис *JSONPlaceholder*, расположенный по адресу <https://jsonplaceholder.typicode.com>, работа будет производиться с сущностью *Todo*. POJO удобнее создавать с помощью готовых веб-сервисов, если структура является сложной. Первым шагом следует описать модель, для этого можно воспользоваться онлайн-сервисами, позволяющими создавать POJO-классы, например <https://www.jsonschema2pojo.org>.

```
public class Todo {  
    @SerializedName("userId")  
    @Expose  
    private Integer userId;  
    @SerializedName("id")  
    @Expose  
    private Integer id;  
    @SerializedName("title")  
    @Expose  
    private String title;  
    @SerializedName("completed")  
    @Expose  
    private Boolean completed;  
    // getters/setters  
}
```

2. Создать интерфейс и указать имя метода. Добавить необходимые параметры. В интерфейсе задаются команды-запросы для сервера. Примеры различного рода запросов с указанием параметров приведены в [документации](#). Запросы размещаются в обобщённом классе *Call* с указанием желаемого типа.

```
public interface ApiService {  
    @GET("todos")  
    Call<List<Todo>> getTodos();  
}
```

В большинстве случаев будет возвращаться объект *Call<T>* с нужным типом (напр. *Call<User>*). В случаях, когда тип ответа не используется в других слоях, то возможно указать *Call<Response>*.

С помощью аннотации указывается тип и структура запросов. Для динамических параметров используются фигурные скобки (*users/{user}/repos*), в которые подставляются нужные значения.

В самой аннотации используется метод, используемый на сервере, а ниже вы можете указать свой вариант (полезно для соответствия стилю вашего кода).

```
@GET("get_all_cats") // команда на сервере
```

Аннотация *@Query* полезна при запросах с параметрами. Например, имеется дополнительный параметр в запросе по получению списка *котов* и его сортировке, который выводит список элементов в отсортированном виде: *http://example.com/api/v1/products/cats?sort=desc*. Данный пример не является сложным, и возможно поместить запрос с параметром в интерфейс без изменений.

```
@GET("products/cats?category=5&sort=desc")
Call<Cats> getAllCats();
```

Если не требуется управлять сортировкой, то её возможно оставить в коде и она будет применяться по умолчанию. Но в запросе имеется ещё один параметр, который отвечает за категорию *котов* (домашние, уличные, породистые), которая может меняться в зависимости от логики приложения. Данный параметр возможно пометить аннотацией и программно управлять в коде.

```
@GET("products/cats?sort=desc")
Call<Cats> getAllCats(@Query("category") int categoryId);
```

Настройка типа сортировки остается постоянной, а категория устанавливается аргументом метода *categoryId*.

```
Call<Cats> getAllCats() = catAPIService.getAllCats(5);
```

Запрос будет иметь следующий вид: <http://example.com/api/v1/products/cats?sort=desc&category=5>. Стоит отметить, что в одном методе возможно указывать несколько *Query*-параметров.

@Path

Запрос может иметь изменяемые части пути. Например, в сервисе *GitHub* поле *username* является конкретным именем пользователя (<https://api.github.com/users/mireq>). В таких случаях используют фигурные скобки в запросе, а в самом методе с помощью аннотации *@Path* указывается имя, которое будет подставляться в путь.

```
@GET("/users/{username}")
Call<User> getUser(
    @Path("username") String userName
);
```

@Headers

Аннотация *@Headers* позволяет указать заголовки запроса.

```
@Headers({"Cache-Control: max-age=640000", "User-Agent: My-App-Name"})
@GET("some/endpoint")
```

@Multipart

Пример аннотации *@Multipart* при загрузке файлов или картинок:

```
@Multipart
@POST("some/endpoint")
Call<Response> uploadImage(@Part("description") String description, @Part("image")
RequestBody image)
```

@Url

Пример аннотации *@Url* для загрузки файла :

```
public interface UserService {
    @GET
    public Call<File> getZipFile(@Url String url);
}
```

3. *Создание и настройка Retrofit*. Для синхронного запроса используется метод *Call.execute()*, для асинхронного - метод *Call.enqueue()*. Объект запроса к серверу создаётся в простейшем случае следующим образом:

```
public static final String BASE_URL = "https://jsonplaceholder.typicode.com/";
```

```
Retrofit retrofit = new Retrofit.Builder()
    .baseUrl(BASE_URL)
    .addConverterFactory(GsonConverterFactory.create())
    .build();
```

В результате возвращается объект *Retrofit*, содержащий базовый URL и конвертор преобразования JSON-данных с помощью *Gson*. Далее в его методе *create()* указывается класс интерфейса с запросами к сайту.

```
apiService = retrofit.create(ApiService.class);
```

После этого возвращается объект *Call* и вызывается метод *enqueue()* (для асинхронного вызова). Запрос будет выполнен в отдельном потоке, а результат придет в *Callback* в *main*-потоке.

```

//creating an api call
Call<Todo> call = service.getTodos();

//making the call
call.enqueue(new Callback<Heroes>() {
    @Override
    public void onResponse(Call< Todo > call, Response<Heroes> response) {

...
    }

    @Override
    public void onFailure(Call< Todo > call, Throwable t) {
        Toast.makeText(getApplicationContext(), t.getMessage(), Toast.LENGTH_LONG).show();
    }
});

```

В результате библиотека *Retrofit* выполнит запрос, получит ответ и произведется разбор ответа. Основная часть работы происходит в методе *onResponse*, ошибки выводятся в *onFailure* (неправильный адрес сервера, некорректные формат данных, неправильный формат класса-модели и т.п). HTTP-коды сервера (например, 404) не относятся к ошибкам.

Метод *onResponse* вызывается даже если запрос был неуспешным. Класс *Response* имеет метод *isSuccessful* для успешной обработки запроса (коды 200xx). В ошибочных ситуациях возможно обработка ошибки в методе *errorBody* класса *ResponseBody*.

Другие полезные методы *Response*.

- **code()** – HTTP-код ответа
- **body()** – сам ответ в виде строки, без сериализации
- **headers()** – HTTP-заголовки
- **message()** – HTTP-статус (или null)
- **raw()** – сырой HTTP-ответ

4. *Создание разметки для одной записи.* Отображение списка будет реализовано с помощью *RecyclerView*, опишем разметку элемента *item.xml*.

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="16dp">
    <TextView
        android:id="@+id/textViewTitle"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textSize="18sp" />
    <CheckBox
        android:id="@+id/checkboxCompleted"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
</LinearLayout>

```

5. Реализация *ViewHolder*

```

public class TodoViewHolder extends RecyclerView.ViewHolder {
    TextView textViewTitle;
    CheckBox checkBoxCompleted;
    public TodoViewHolder(@NonNull View itemView) {
        super(itemView);
        textViewTitle = itemView.findViewById(R.id.textViewTitle);
        checkBoxCompleted = itemView.findViewById(R.id.checkBoxCompleted);
    }
}

```

6. Реализация Adapter.

```

public class TodoAdapter extends RecyclerView.Adapter<TodoViewHolder> {
    private LayoutInflater inflater;
    private List<Todo> todos;

```

```

    public TodoAdapter(Context context, List<Todo> todoList) {
        this.inflater = LayoutInflater.from(context);
        this.todos = todoList;
    }

```

```

    @NonNull
    @Override
    public TodoViewHolder onCreateViewHolder(@NonNull ViewGroup parent, int
viewType) {
        View view = inflater.inflate(R.layout.item, parent, false);
        return new TodoViewHolder(view);
    }

```

```

    @Override
    public void onBindViewHolder(@NonNull TodoViewHolder holder, int position) {
        Todo todo = todos.get(position);
        holder.textViewTitle.setText(todo.getTitle());
        holder.checkBoxCompleted.setChecked(todo.getCompleted());
    }
    @Override
    public int getItemCount() {
        return todos.size();
    }
}

```

7. В MainActivity произведем конфигурацию Retrofit, выполним запрос и отобразим полученные данные в RecyclerView.

```

public class MainActivity extends AppCompatActivity {
    public static final String TAG = "MainActivity";
    public static final String BASE_URL =
        "https://jsonplaceholder.typicode.com/";
    private RecyclerView recyclerView;
    private TodoAdapter todoAdapter;
    private ApiService apiService;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        // ...

        recyclerView = findViewById(R.id.recyclerView);
        recyclerView.setLayoutManager(new LinearLayoutManager(this));
        Retrofit retrofit = new Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build();
        apiService = retrofit.create(ApiService.class);
        Call<List<Todo>> call = apiService.getTodos();
        call.enqueue(new Callback<List<Todo>>() {
            @Override
            public void onResponse(Call<List<Todo>> call,
                Response<List<Todo>> response) {
                if (response.isSuccessful() && response.body() != null) {
                    List<Todo> todos = response.body();
                    todoAdapter = new TodoAdapter(MainActivity.this, todos);
                    recyclerView.setAdapter(todoAdapter);
                } else {
                    Log.e(TAG, "onResponse: " + response.code());
                }
            }
            @Override
            public void onFailure(Call<List<Todo>> call, Throwable t) {
                Log.e(TAG, "onFailure: " + t.getMessage());
            }
        });
    }
}

```

Задание. В текущее приложение добавить отправку запроса на обновление Todo при обновлении состояния CheckBox.

2 PICASSO

Одним из частых сценариев применения сетевых запросов является загрузка изображений из интернета для показа их пользователю. Данную задачу возможно решить с использованием библиотеки *Picasso*, разработанную компанией *Square Open Source*, которая делает процесс работы с изображениями удобным и эффективным. Основной функцией является загрузка изображений из интернета прямо в виджеты (например *ImageView*), данный процесс осуществляется асинхронно, что предотвращает блокировку пользовательского интерфейса при загрузке больших изображений. Библиотека автоматически обрабатывает ошибки, такие как проблемы с соединением или недоступность изображения.

Основные возможности *Picasso*:

1. Асинхронная загрузка изображений: изображения загружаются в фоновом потоке, что предотвращает блокировку пользовательского интерфейса.
2. Кэширование: изображения кэшируются в памяти и на диске для ускорения загрузки при повторных запросах.
3. Автоматическая обработка ошибок: *Picasso* может автоматически отображать изображения по умолчанию, если исходное изображение не может быть загружено.
4. Масштабирование и преобразование изображений: библиотека поддерживает масштабирование, обрезку и другие операции с изображениями (например, круглый стиль, создание эффектов).
5. Поддержка различных источников: изображения можно загружать не только с URL, но и из локальных ресурсов, файлов, битмапов и т. д.

Важной особенностью является автоматическое кэширование изображений, что ускоряет повторную загрузку уже полученных изображений, улучшает производительность приложения и снижает количество сетевых запросов, это особенно полезно при работе в условиях ограниченных сетевых ресурсов. Возможно задать изображение-заглушку, которое будет отображаться пока изображение не загрузится, в случае возникновения ошибки вместо изображения можно отобразить резервное изображение. *Picasso* также позволяет выполнять трансформации изображений, например изменять размер, обрезать, вращать.

Загрузка изображений возможна через метод *load()*, который позволяет загрузить изображение по URL, метод *into()* устанавливает загруженное изображение в *ImageView*. Методы *placeholder()* и *error()* позволяют указать изображения-заглушки, которые будут отображаться при загрузке или ошибке загрузки изображения соответственно. Настройка параметров отображения возможна с помощью отдельных методов, таких как *resize*, *fit*, *centerCrop*, *centerInside*, *transform*.

Пример использования *Picasso*:

1. Добавление зависимости в **build.gradle**:
implementation 'com.squareup.picasso:picasso:2.8'

2. Загрузка изображения в ImageView

```
Picasso.get()  
.load("https://example.com/image.jpg")  
.into(imageView);
```

3. Загрузка изображения с обработкой ошибок:

```
Picasso.get()  
.load("https://example.com/image.jpg")  
.error(R.drawable.error_image) // изображение, если загрузка не удалась  
.placeholder(R.drawable.placeholder) // изображение, пока загружается  
.into(imageView);
```

4. Загрузка изображения в ImageView с заданными размерами::

```
Picasso.get()  
.load("https://example.com/image.jpg")  
.resize(100, 100) // Установить размер  
.centerCrop() // Обрезать изображение  
.into(imageView);
```

Преимущества использования Picasso:

- Простота в использовании: с минимальными усилиями можно начать загружать изображения в приложение.
- Управление ресурсами: кэширование изображений помогает эффективно использовать память и снижает нагрузку на сеть.
- Гибкость: Picasso поддерживает множество вариантов обработки изображений, таких как округление, изменение размера, создание эффектов и т.д.

Недостатки:

- Picasso не предоставляет таких продвинутых возможностей, как другие библиотеки, например Glide или Coil, которые могут иметь более быструю загрузку, поддержку анимаций или оптимизацию под современные устройства и API.

Тем не менее, Picasso — это стабильное и простое решение для большинства приложений, которым не нужны сложные или специфичные требования к изображению.

Задание. Измените разметку предыдущего задания и добавьте изображение. Добавьте загрузку нескольких изображений из интернета и их отображение с помощью Picasso, добавьте возможность настройки параметров отображения.

3 КОНТРОЛЬНОЕ ЗАДАНИЕ

В ходе выполнения практического задания № 1 были сформулированы use-case возможности проекта. Приложение должно удовлетворять следующим функциональным требованиям:

1. Авторизация в приложении.
2. Взаимодействия с каким-либо внешним сервисом, возвращающим данные о стоимости валюты, погоды и т.д. в виде json-формата. Возможно использовать сервисы типа <https://www.wiremock.io/>, <https://mockapi.io/>, <https://github.com/typicode/json-server>, <https://thecatapi.com/> и т.д.
3. Сохранение данных в БД.
4. Отображение списка каких-либо сущностей с изображениями.
5. Отображение страницы сущности.
6. Различные возможности у авторизованного и гостевого пользователя.
7. Использование обученной модели для распознавания изображений, аудио, маски и т.д. (нужно произвести анализ существующих обученных моделей TensorFlow Lite, которые желаете использовать в приложении).

Задание. Требуется открыть приложение из практического занятия №1. Требуется открыть приложение из практической работы №1. Реализовать получение сущностей из сети с помощью Retrofit, обработать возможные ошибки, реализовать отображение изображений с помощью Picasso, Coil или Glide.